بسمه تعالی

مجموعه ها (set)

مجموعه ها برای ذخیره چند مقدار در یک متغیر استفاده میشوند. مجموعه یکی از چهار نوع داده پایتون است که برای ذخیرهسازی انواع دادهها مثل اعداد صحیح، اعداد اعشاری، رشته ها، بولین ها و توپل ها به صورت مجموعه ای به کار می روند.

نحوه ساختن مجموعه یا set

برای ساخت مجموعه، از علامت آکولاد استفاده می‌کنیم. ابتدا متغیری را تعریف کرده و سپس داخل آکولاد مجموعه ای از عناصر مشخص و از پیش تعیین شده را تعریف می‌کنیم.

نکته: مجموعه یا set ،ترتیب نمایش عناصر را تضمین نمی‌کند و به اصطلاح Unordered می‌باشند. به عنوان مثال، اگر مجموعه ی زیر را در نظر بگیریم، با هر بار اجرای برنامه ممکن است خروجی برنامه با ترتیب مختلفی نمایش داده شود.

```
fruits = {"apple", "banana", "orange", "mango"}

print(fruits)
```

ممکن است خروجی به شکل زیر باشد:

{'banana', 'apple', 'mango', 'orange'}

و در اجرای دیگری ممکن است ترتیب متفاوتی نمایش داده شود:

{'mango', 'orange', 'banana', 'apple'}

بنابراین در set نباید به ترتیب عناصر وابسته باشیم.همچنین هر عنصر فقط یک بار در مجموعه قرار می‌گیرد؛ یعنی Set عناصر تکراری را نگه نمی‌دارد.

استفاده از متد add برای اضافه کردن عنصر جدید به set

برای اضافه کردن یک عنصر جدید به Set در پایتون، از متد add()استفاده می‌کنیم.

```
fruits = {"apple", "banana", "orange"}

fruits.add("mango")

print(fruits)
```

در اینجا " "mangoبه مجموعه اضافه می‌شود.

نکته: اگر عنصری که با add()اضافه می‌کنیم از قبل در مجموعه وجود داشته باشد، دوباره اضافه نمی‌شود؛ چون در Set عناصر تکراری وجود ندارند.

استفاده از متد remove برای حذف عنصر از set

برای حذف یک عنصر از Set در پایتون، از متد remove() استفاده می‌کنیم.

```
fruits = {"apple", "banana", "orange"}

fruits.remove("banana")

print(fruits)
```

در اینجا " "bananaاز مجموعه حذف می‌شود.

نکته: اگر عنصری که می‌خواهیم حذف کنیم در Set وجود نداشته باشد، متد remove()خطا ایجاد می‌کند.

استفاده از in برای بررسی وجود عنصر set

برای بررسی اینکه یک عنصر در Set وجود دارد یا نه، از عملگر in استفاده می‌کنیم.

```
fruits = {"apple", "banana", "orange"}

if "banana" in fruits:
    print("Banana is in the set")
```

خروجی

Banana is in the set

اگر عنصر موردنظر در Set وجود داشته باشد، شرط in مقدار True و اگر وجود نداشته باشد مقدار False برمی‌گرداند.

مثال ساده تر

```
fruits = {"apple", "banana", "orange"}

print("apple" in fruits)
print("mango" in fruits)
```

خروجی

True

False

مثال: تکه کدی بنویسید که بررسی کند آیا عنصر اپل در مجموعه وجود دارد یا ندارد.

```
fruits = {"apple", "banana", "orange"}

if "apple" in fruits:
    print("Apple is in the set")
else:
    print("Apple is not in the set")
```

فعالیت

مجموعه‌ای از میوه‌ها وجود دارد. برنامه‌ای بنویسید که کاربر نام یک میوه‌ای را وارد کند و اگر آن میوه در set بود، پیام دهد که آن میوه درون مجموعه موجود است و set نهایی را چاپ کند.

```
fruits = {"apple", "banana", "orange", "mango"}

fruit = input("Enter a fruit: ")

if fruit in fruits:
    print("This fruit is in the set")
else:
    print("This fruit is not in the set")

print(fruits)
```

اگر کاربر apple وارد کند، پیام موجود بودن آن نمایش داده می‌شود و در انتها set نهایی چاپ می‌شود.

نوع داده Dictionary

دیکشنری (Dictionary) نوعی داده در پایتون است که اطلاعات را به صورت کلید و مقدار (Key-Value) ذخیره می‌کند. هر کلید به یک مقدار مربوط است.

فرمت کلی:

```
dictionary_name = {
   "key1": "value1",
   "key2": "value2",
   "key3": "value3"
}
```

مثال: یک نمونه از دیکشنری بنویسید که شامل سه آیتم کلید و مقدار به نام Cardict باشد.

```
Cardict = {
    "brand": "BMW",
    "model": "X5",
    "year": 2024
}
print(Cardict)
```

در این مثال دیکشنری Cardict شامل سه آیتم کلید و مقدار است:

brand → BMW

model → X5

year → 2024

استفاده از براکت در نمایش محتوای دیکشنری

مثال: یک دیکشنری به نام Person با سه کلید و مقدار تعریف می‌کنیم و سپس هر مقدار را با استفاده از براکت [] در print نمایش می‌دهیم:

```
Person = {
    "name": "Ali",
    "age": 20,
    "city": "Baku"
}

print(Person["name"])
print(Person["age"])
print(Person["city"])
```

خروجی:

Ali

20

Baku

برای دسترسی به مقدار هر کلید، نام کلید را داخل براکت [] قرار می‌دهیم.

نکته مهم درباره Dictionary

اگر هر داده دارای عنوان مشخصی باشد، استفاده از دیکشنری بهتر از لیست است؛ زیرا در دیکشنری می‌توان با استفاده از کلید (Key)مستقیماً به مقدار (Value)دسترسی پیدا کرد و نیازی به دانستن اندیس (Index) نداریم.

```
student = {
    "name": "Ali",
    "grade": 18
}

print(student["name"])
print(student["grade"])
```

خروجی :

ali

18

در اینجا به جای اینکه بدانیم نام دانش‌آموز در اندیس ۰ و نمره در اندیس ۱ قرار دارد، مستقیماً با کلیدهای " "nameو " "grade به اطلاعات دسترسی پیدا می‌کنیم.

فعالیت:

یک دیکشنری به نام book بسازید که اطلاعات یک کتاب را نگه دارد. Title: Python Basic.

```
book = {
    "Title": "Python Basic",
    "Author": "reza",
    "Pages": 250
}

print(book["Author"])
print(book["Pages"])
```

نکته: در دیکشنری کلیدها باید متفاوت باشند، اما می‌توانند مقدار یکسان داشته باشند.

```
students = {
    "Ali": 18,
    "Reza": 18,
    "Sara": 20
}
print(students)
```

در این مثال، کلیدهای " "Aliو " "Rezaمتفاوت هستند، اما مقدار هر دو ۱۸ است.

نکته: در Dictionary هر کلید باید یکتا (Unique) باشد. اگر یک کلید چند بار تکرار شود، مقدار جدید، مقدار قبلی را بازنویسی (Overwrite) می‌کند.

```
car_dict = {
    "brand": "BMW",
    "model": "X5",
    "year": 2020,
    "color": "Black",
    "brand": "Toyota"
}

print(car_dict)
```

خروجی

{'brand': 'Toyota', 'model': 'X5', 'year': 2020, 'color': 'Black'}

اگرچه در کد brand دو بار نوشته شده است، در دیکشنری فقط یک کلید brand باقی می‌ماند.

مقدار " "BMWبا مقدار جدید " "Toyotaجایگزین می‌شود.

بنابراین این دیکشنری در نهایت ۴ آیتم دارد، نه ۵ آیتم.

نکته: برای تعریف یک دیکشنری خالی با نام user می‌توانیم از {} استفاده کنیم:

```
user = {}

print(user)
```

نکته: برای اضافه کردن یک آیتم جدید به دیکشنری، کافی است یک کلید جدید را مقداردهی کنیم.

```
user = {}

user["name"] = "Ali"
user["age"] = 20

print(user)
```

خروجی

{'name': 'Ali', 'age': 20}

در اینجا با user["name"] و user["age"] دو آیتم جدید به دیکشنریِ خالی اضافه شده‌اند

فعالیت:

یک دیکشنری به نام Product با کلیدهای Name و Price بسازید، قیمت محصول درون دیکشنری را تغییر دهید و سپس دیکشنری را چاپ کنید.

```
Product = {
    "Name": "Laptop",
    "Price": 500
}

Product["Price"] = 700

print(Product)
```

خروجی

{'Name': 'Laptop', 'Price': 700}

در اینجا قیمت محصول از ۵۰۰ به ۷۰۰ تغییر کرده است.

فعالیت:

یک دیکشنری خالی با نام user بسازید، از ورودی مقادیری برای کلیدهای username و password و age را بگیرید و داخل دیکشنری ذخیره و در پایان کل اطلاعات کاربر را چاپ کنید.

```
user = {}
```

```
user["username"] = input("Enter username: ")
user["password"] = input("Enter password: ")
user["age"] = input("Enter age: ")

print(user)
```

مثلا اگر کاربر موارد زیر را وارد کند:

Enter username: ali

Enter password: 1234

Enter age: 20

خروجی

{'username': 'Ali', 'password': '1234', 'age': '20'}

نکته: اگر بخواهیم age به صورت عدد ذخیره شود، بهتر است بنویسیم:

```
user["age"] = int(input("Enter age: "))
```

حذف عنصر از دیکشنری

برای حذف یک عنصر از دیکشنری ، می‌توانیم از دستور del استفاده کنیم. در این روش، کلید موردنظر را داخل براکت [] قرار می‌دهیم.

```
user = {
    "username": "Ali",
    "age": 20,
    "city": "Baku"
}

del user["age"]

print(user)
```

خروجی

{'username': 'Ali', 'city': 'Baku'}

در مثال فوق ، کلید " "age و مقدار مربوط به آن، یعنی ۲۰، از دیکشنری حذف شده است.

استفاده از متد pop() برای حذف یک عنصر از دیکشنری

متد pop()برای حذف یک عنصر بر اساس نام کلید (Key) استفاده می‌شود.شکل کلی استفاده از این متد به صورت زیر است.

dictionary.pop(key)

```
user = {
    "username": "Ali",
    "age": 20,
    "city": "Baku"
}

user.pop("age")

print(user)
```

خروجی

{'username': 'Ali', 'city': 'Baku'}

در این مثال، pop("age") کلید age و مقدار مربوط به آن یعنی ۲۰ را از دیکشنری حذف می‌کند

فعالیت:

یک دیکشنری به نام inventory بسازید که شامل موارد زیر باشد.

inventory = { "bag": 5, "ball": 8, "laptop": 3,"mouse": 10,”flash memory”:14}

وجود کالایی به نام bag را بررسی و در صورت وجود پیام مناسبی چاپ شود که نشان‌دهنده وجود کالا باشد. سپس آن آیتم را با متد pop حذف کنید. مقدار حذف شده را چاپ کنید. اگر کالا وجود نداشت پیام مبنی بر عدم وجود آن نمایش داده شود. سپس دیکشنری جدید را چاپ کنید.

```
inventory = {
    "bag": 5,
    "ball": 8,
    "laptop": 3,
    "mouse": 10,
    "flash memory": 14
}

if "bag" in inventory:
    print("Bag is available")

    removed = inventory.pop("bag")
    print("Removed value:", removed)
else:
    print("Bag is not available")

print("New inventory:", inventory)
```

خروجی

Bag is available

Removed value: 5

New inventory: {'ball': 8, 'laptop': 3, 'banana': 10, 'flash memory': 14}

فعالیت –پروژه بازی Quiz

در این بازی ابتدا منویی که شامل انبوهی سوالات است نمایش داده می‌شود و کاربر سوال مورد علاقه‌اش را انتخاب می‌کند و سپس به سوالات پاسخ می‌دهد. در صورت پاسخ صحیح یک امتیاز دریافت می‌کند و در غیر این صورت امتیازی دریافت نمی‌کند. در صورت تمام شدن سوالات امتیاز کاربر نمایش داده می‌شود

```
menu = {
    "1": "Math Quiz",
    "2": "Geography Quiz",
    "3": "Exit"
}

math_quiz = {
    "2 + 2 = ?": "4",
    "5 * 3 = ?": "15",
    "10 - 7 = ?": "3"
}

geo_quiz = {
    "Capital of Iran?": "Tehran",
    "Capital of France?": "Paris",
    "Capital of Japan?": "Tokyo"
}

print("Welcome to Quiz Game!")

print("Menu:")

for key in menu:
    print(key, "-", menu[key])

choice = input("Select an option: ")

score = 0

if choice == "1":
    print("\nMath Quiz")

    for question in math_quiz:
        answer = input(question + " ")
```

```
        if answer == math_quiz[question]:
            print("Correct!")
            score += 1
        else:
            print("Wrong!")

elif choice == "2":
    print("\nGeography Quiz")

    for question in geo_quiz:
        answer = input(question + " ")

        if answer == geo_quiz[question]:
            print("Correct!")
            score += 1
        else:
            print("Wrong!")

elif choice == "3":
    print("Exiting the game...")

else:
    print("Invalid option")

if choice in ["1", "2"]:
    print("\nYour final score is:", score)
```

در مثال فوق بخش های جدید Quiz، مثلا Science و sport را اضافه کنید. برای جواب غلط یک پیام توضیح بدهید. امکان انتخاب چند بخش به صورت پشت سر هم را اضافه کنید.

در تکمیل پروژه فوق بخش‌های Science و Sport اضافه شده‌اند، برای پاسخ اشتباه توضیح صحیح نمایش داده می‌شود و کاربر می‌تواند چند Quiz را پشت سر هم انتخاب کند. با انتخاب ۳ بازی تمام می‌شود.

```
menu = {
    "1": "Math Quiz",
    "2": "Geography Quiz",
    "3": "Science Quiz",
    "4": "Sport Quiz",
    "5": "Exit"
}

math_quiz = {
    "2 + 2 = ?": "4",
    "5 * 3 = ?": "15",
    "10 - 7 = ?": "3"
}

geo_quiz = {
    "Capital of Iran?": "Tehran",
    "Capital of France?": "Paris",
    "Capital of Japan?": "Tokyo"
}

science_quiz = {
    "What planet is known as the Red Planet?": "Mars",
    "What is H2O?": "Water",
    "How many legs does a spider have?": "8"
}

sport_quiz = {
    "How many players are on a soccer team on the field?": "11",
    "Which sport uses a racket and a shuttlecock?": "Badminton",
    "How many rings are on the Olympic flag?": "5"
}

print("Welcome to Quiz Game!")

total_score = 0

while True:
    print("\nMenu:")

    for key in menu:
        print(key, "-", menu[key])
```

```
    choice = input("Select an option: ")

    if choice == "1":
        quiz = math_quiz
        print("\nMath Quiz")

    elif choice == "2":
        quiz = geo_quiz
        print("\nGeography Quiz")

    elif choice == "3":
        quiz = science_quiz
        print("\nScience Quiz")

    elif choice == "4":
        quiz = sport_quiz
        print("\nSport Quiz")

    elif choice == "5":
        print("Exiting the game...")
        break

    else:
        print("Invalid option!")
        continue

    score = 0

    for question in quiz:
        answer = input(question + " ")

        if answer.lower() == quiz[question].lower():
            print("Correct!")
            score += 1
        else:
            print("Wrong!")
            print("The correct answer is:", quiz[question])
    print("Your score in this quiz is:", score, "out of", len(quiz))
    total_score += score

    print("Your total score is:", total_score)
```

Science Quiz و Sport Quiz به منو اضافه شده‌اند.

با while True کاربر می‌تواند چند بخش را پشت سر هم انتخاب کند.

با انتخاب ۵ برنامه متوقف می‌شود.

در صورت پاسخ اشتباه، جواب صحیح نمایش داده می‌شود.

total_score امتیاز تمام بخش‌هایی را که کاربر بازی کرده، جمع می‌کند.

توپِل (Tuple)

تاپل در پایتون دنباله‌ای از داده‌های ترتیبی و غیرقابل تغییر است. به کمک تاپل‌ها می‌توانید چندین مقدار از نوع‌های داده‌های مختلف را در یک متغیر ذخیره و روی آنها عملیات انجام دهید. این ساختار داده را زمانی استفاده کنید که بخواهید داده‌ها در طول برنامه تصادفاً توسط خود شما و یا سایر برنامه‌نویسان هم‌تیمی شما تغییر نکنند. شکل کلی ایجاد تاپل به صورت زیر می‌باشد.

tuple_name = (value1, value2, value3)

مثال: یک تاپل با سه عضو برای نگهداری نام کاربری مدیران ارشد شرکت ایجاد و سپس آن را چاپ کنید.

```
admins = ("Super Admin", "Manager Admin", "Teacher Admin")

print(admins)
```

خروجی

('Super Admin', 'Manager Admin', 'Teacher Admin')

مثال: در تکه کد زیر از یک تاپل برای نگهداری نام روزهای هفته استفاده شده است، زیرا نام روزهای هفته تغییرپذیر نیستند. بنابراین این نوع ساختار داده مناسب نگهداری این مقادیر میباشد.

```
days = (
    "Saturday",
    "Sunday",
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday"
)

print(days)
```

خروجی

('Saturday', 'Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday')

دسترسی به عناصر توپل:

مانند لیست‌ها، عناصر تاپل هم دارای ایندکس هستند و دسترسی به عنصر با شماره ایندکس آن انجام می‌شود.

مثال: در این برنامه ابتدا یک تاپل به نام name_port برای نگهداری نام درگاه‌های ورودی یک سیستم رایانه‌ای ایجاد و سپس عنصر سوم آن نمایش داده شده است. دقت کنید ایندکس عناصر در توپل از صفر شروع می‌شود.

```
name_port = ("USB", "HDMI", "Ethernet", "VGA")

print(name_port[2])
```

خروجی

Ethernet

در این مثال:

name_port[0] → USB

name_port[1] → HDMI

name_port[2] → Ethernet

name_port[3] → VGA

نکته مهم: عناصر Tuple تغییرناپذیر (Immutable) هستند. بنابراین نمی‌توانیم بعد از ایجاد Tuple مقدار یک عنصر را تغییر دهیم.به عنوان مثال تکه کد زیر با خطا مواجه می شود،زیرا مقادیر عناصر Tuple قابل تغییر نمی باشد.

```
name_port = ("USB", "HDMI", "Ethernet", "VGA")

name_port[2] = "LAN"
```

آیا می‌توانیم تعداد عناصر یک تاپل را تغییر دهیم یا نه؟ برنامه ای بنویسید که نتیجه این سوال را به شما نشان دهد. به نظر شما مزیت اصلی تاپل چیست که برنامه‌نویس را ملزم می‌کند از آن استفاده کند.

خیر، نمی‌توانیم تعداد عناصر یک Tuple را تغییر دهیم؛ چون Tuple یک ساختار داده‌ی غیرقابل تغییر (Immutable) است.

```
numbers = (10, 20, 30)

print("Original tuple:", numbers)

numbers.append(40)

print("New tuple:", numbers)
```

این برنامه با خطا مواجه می‌شود، چون متد append()برای Tuple وجود ندارد.

حذف tuple

در پایتون امکان حذف عناصر یک **tuple** وجود ندارد، ولی می‌توانید کل یک **tuple** را با استفاده از دستور **del** حذف کنید.

مثال: در این برنامه با استفاده از دستور **del** تاپل **days** حذف شده است. بعد از اجرای آن هم پیغامی مبنی بر تعریف **tuple days** نمایش داده می‌شود.

در واقع در **Tuple** نمی‌توان یک عنصر خاص را حذف کرد، اما می‌توان کل **Tuple** را با دستور **del** حذف کرد.

```
days = ("Saturday", "Sunday", "Monday", "Tuesday", "Wednesday",
"Thursday", "Friday")

del days

print("Tuple days has been deleted.")
```

خروجی

Tuple days has been deleted.

بعد از اجرای **del days**، دیگر **Tuple** با نام **days** وجود ندارد و اگر بنویسیم:

```
print(days)
```

برنامه خطا می‌دهد، چون متغیر **days** حذف شده است.